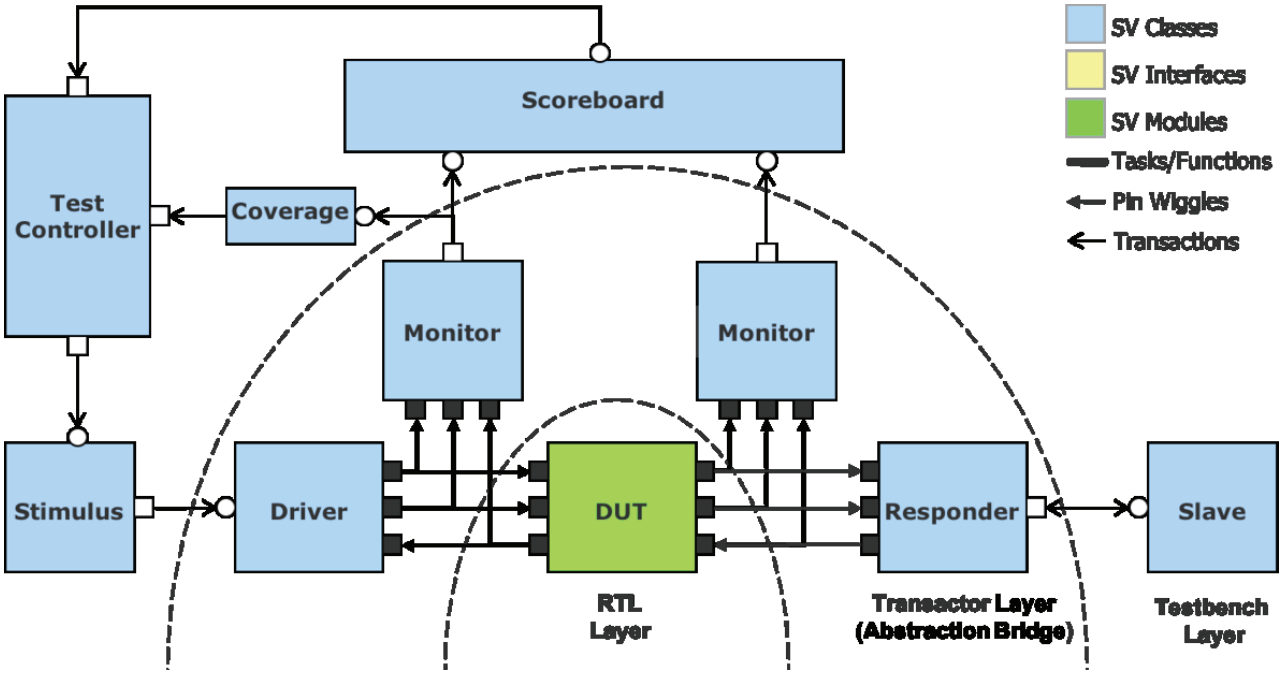
Testbench Architecture

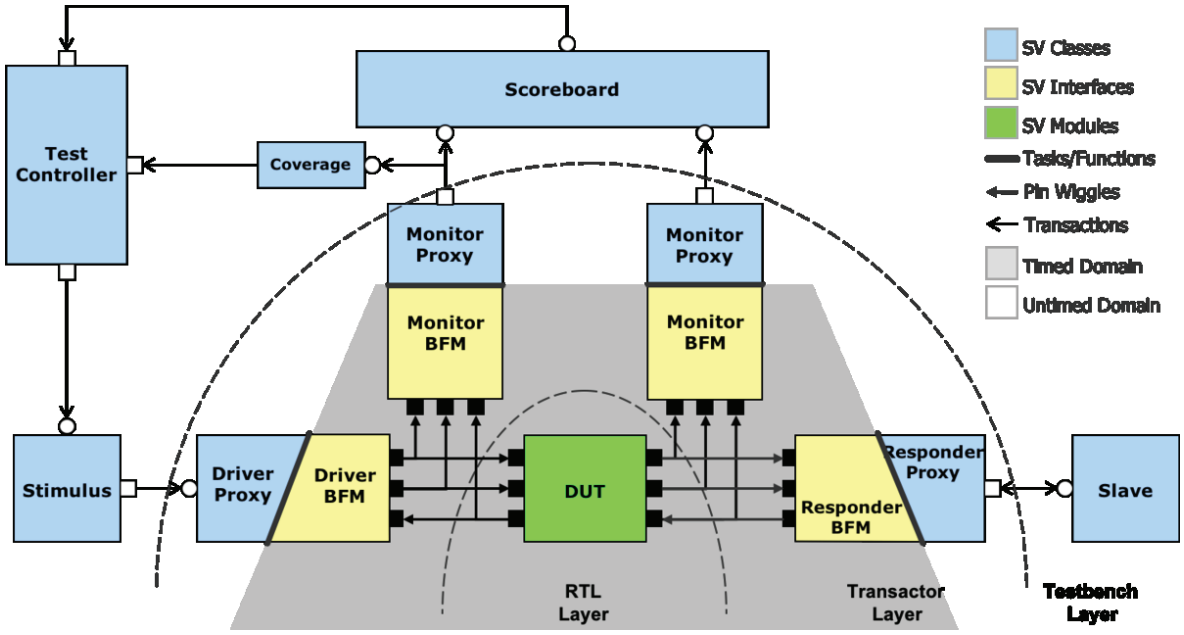
Testbench Architecture

UVM Testbench Architecture [1]

A UVM testbench is built using SystemVerilog (dynamic) class objects interacting with SystemVerilog (static) interfaces and modules in a structured hierarchy. The hierarchy is composed of layers of functionality. At the center of the testbench is the Design Under Test (DUT). It is connected to a layer of transactors (drivers, monitors, responders). The transactors communicate with the DUT at the pin level by driving and sampling DUT signals, and with the rest of the UVM testbench by passing transaction objects. They convert data between pins and transactions, i.e. from/to signal to/from transaction level. The testbench layer above the transactor layer consists of components that interact exclusively at the transaction level, such as scoreboards, coverage collectors, stimulus generators, etc.

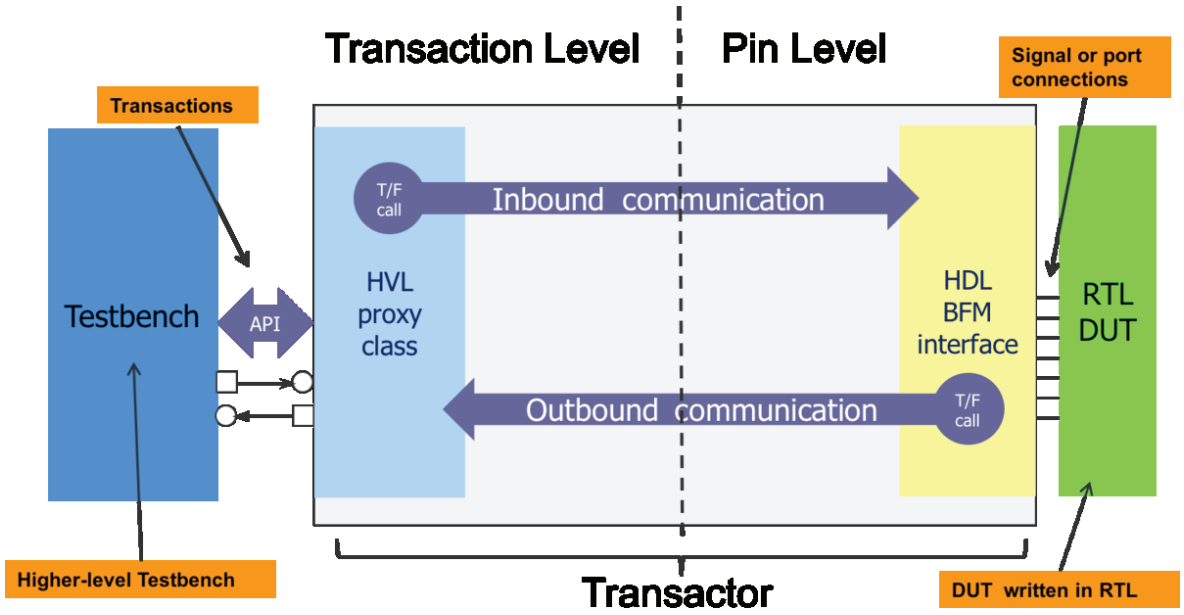


The transactor and testbench layers are conventionally built purely from SystemVerilog classes. However, that construction style limits portability by only targeting a SystemVerilog simulator. A somewhat alternate style, or architecture, that utilizes SystemVerilog classes as well as SystemVerilog interfaces, can increase portability between execution engines. Taking advantage of both these SystemVerilog constructs, a natural split can be made in the transactor layer between transaction level communication grouped on one side separate from timed, signal level communication grouped on the other side.



Partitioned Transactors [1]

The separation of different abstraction levels grouped into like functionality leads to a Dual Top partitioned testbench architecture where one top level contains all of the timed, signal level code in a so-called HDL domain, while the other so-called HVL/TB top contains all of the transaction level testbench domain code. Since transactors typically lie in both domains, they must also be paritioned into two parts. These two parts are a BFM interface and a proxy class. The BFM interface handles signal level code while the proxy class handles anything else that a conventional transactor would do. The BFM and proxy communicate with each other through function and task calls.



While this dual top testbench architecture enables portability [1], it also reduces modeling flexibility to a degree primarily because the signal level code is placed into a SystemVerilog interface instead of a class. SystemVerilog classes offer powerful object-oriented capabilities including inheritance and polymorphism that SystemVerilog interfaces forego.

The UVM Testbench Build and Connection Process

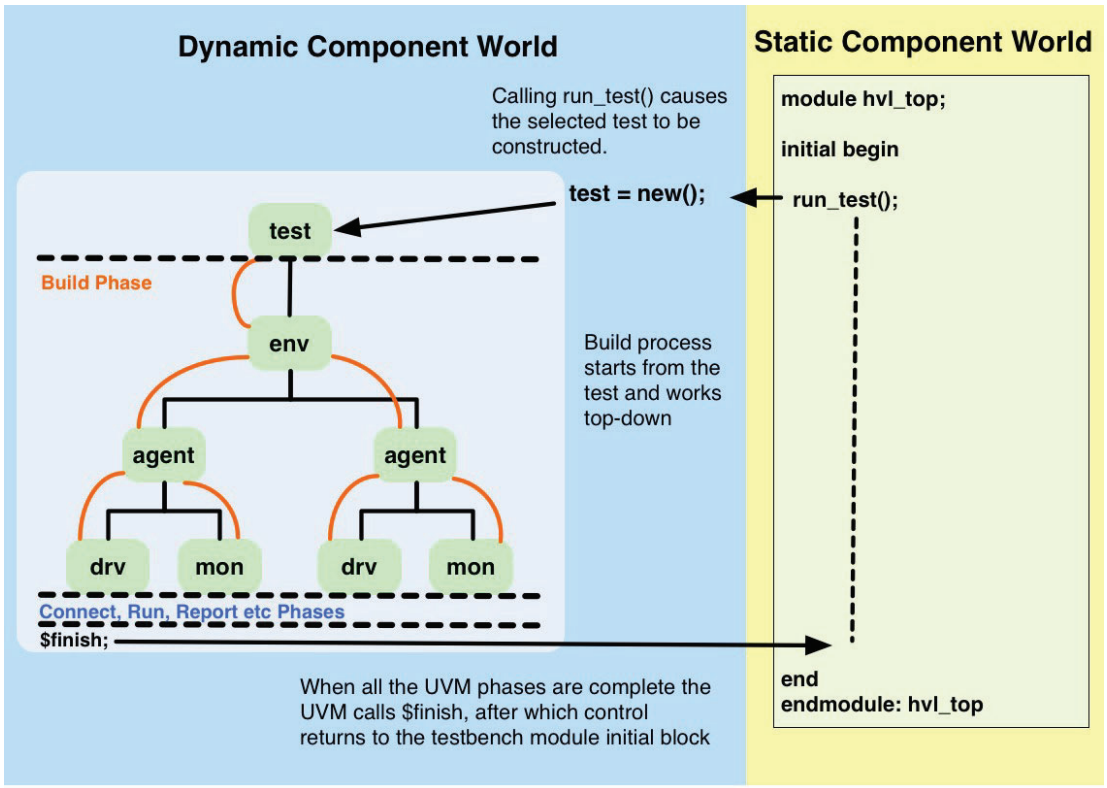
The process to configure and build all the layers of a dual top portable testbench is described in the article on building UVM testbenches. Examples are provided to show how to build a block-level testbench and how to integrate multiple block-level testbenches into higher-level testbench.

References

[1] https://verificationacademy.com/patterns-library/implementation-patterns/environment-patterns/dual-domain-hierarchy-pattern

Building a UVM Testbench

The first phase of a UVM testbench is the build phase. During this phase, the *uvm_component* classes that make up the testbench hierarchy are constructed into objects. The construction process works top-down with each level of the hierarchy constructed and configured before the next level down (sometimes also referred to as deferred construction).



The UVM Build flow in the context of the testbench

The UVM testbench is activated when the run_test() method is called in an initial block of the HVL top level module. This UVM static method takes a string argument that defines the test to be run by name and constructs it via the UVM factory. The UVM infrastructure then starts the build phase by calling the build method of the test class.

During the execution of the test's build phase, the various testbench component configuration objects are prepared and the virtual interfaces in these configuration objects are assigned to the associated testbench interfaces. The configuration objects are then put into the UVM configuration database for this test. Subsequently, the next level of hierarchy is built.

At the next level of hierarchy the corresponding configuration object prepared at the previous level is retrieved and further configuration may take place. Before this configuration object is used to guide the construction and configuration of the next level of hierarchy, it may be modified at the current level of hierarchy. Such conditional

construction thus affects the topology or hierarchical structure of the testbench.

The build phase works top-down and so the process is repeated for each successive level of the testbench hierarchy until the leaf level is reached.

After the build phase completes, the connect phase commences to make all inter-component connections. Contrary to the build phase, the connect phase works bottom-up from the leafs to the top of the testbench hierarchy. Following the connect phase, the rest of the UVM phases run to completion (also bottom-up) upon which control is passed back to the testbench module.

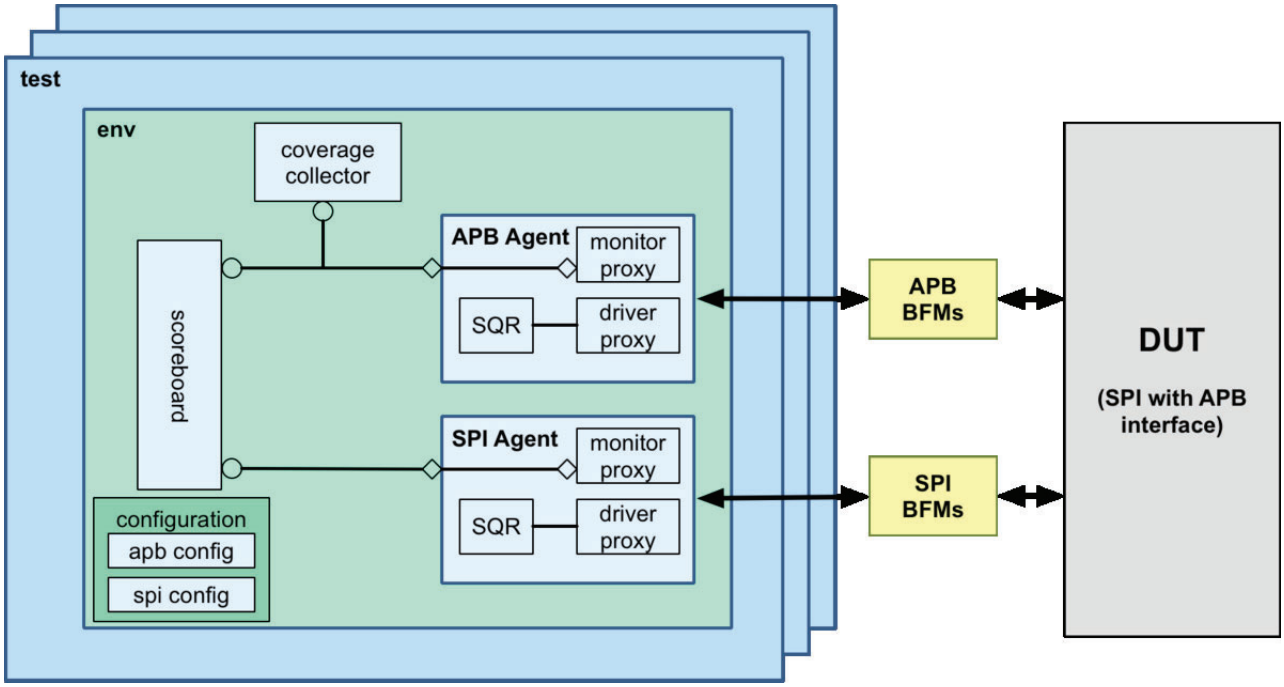
The Test is The Starting Point for The Build Process

The build process for a UVM testbench starts from the test class and works top-down. The test class build method is the first one called at the build phase and it (i.e. the method implementation) determines what gets built in a UVM testbench. Its function is to:

- Set up any factory overrides so that configuration objects or component objects are created as derived types as needed
- Create and configure the configuration objects required by the various sub-components
- Assign the virtual interface handles put into configuration space by the HDL testbench module
- Build up an encapsulating env configuration object and include it into the configuration space
- Build the next level down, usually the top-level env, in the testbench hierarchy

For a given verification environment most of the work done in the build method will be the same for all tests, and so it is recommended that a test base class is created which can be easily extended for each of the test cases.

Shown below is a block level verification environment to help explain concretely how the test build process works. This is an environment for a SPI master interface DUT, which contains two agents, one for its APB bus interface and one for its SPI interface. A detailed account of the build and connect phases for this example is found in the Block Level Testbench Example article.



Factory Overrides

The UVM factory allows a UVM class to be substituted with another derived class at the point of construction. This can be useful for specializing (i.e. customizing or extending) component behavior or a configuration object. The factory override must be specified before the target object is constructed, so it is convenient to do it at the start of the build process.

Sub-Component Configuration Objects

Each grouping component like an agent or env should have a configuration object that defines its structure and behavior. These configuration objects should be created in the build method of the test and implemented to fit the requirements of the test case. If the configuration of a sub-component is either complex or likely to change, it is advised to add a virtual function implementing the basic (or default) configuration handling, which can then be overwritten in test cases extending from the base test class by overloading the virtual function.

```
//
// Class Description:
//
//
class spi_test_base extends uvm_test;

// UVM Factory Registration Macro
//
`uvm_component_utils(spi_test_base)

//------------------------------------------
// Data Members
//------------------------------------------

//------------------------------------------
// Component Members
//------------------------------------------
// The environment class
spi_env m_env;

// Configuration objects
spi_env_config m_env_cfg;
apb_agent_config m_apb_cfg;
spi_agent_config m_spi_cfg;

//------------------------------------------
// Methods
//------------------------------------------

// Standard UVM Methods:
extern function new(string name = "spi_test_base", uvm_component parent = null);
extern function void build_phase( uvm_phase phase );
extern virtual function void configure_apb_agent(apb_agent_config cfg);
extern function void set_seqs(spi_vseq_base seq);

endclass: spi_test_base
```

```
function spi_test_base::new(string name = "spi_test_base", uvm_component parent = null);
  super.new(name, parent);
endfunction

// Build the env, create the env configuration
// including any sub configurations
function void spi_test_base::build_phase(uvm_phase phase);
  // env configuration
  m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
  // APB configuration
  m_apb_cfg = apb_agent_config::type_id::create("m_apb_cfg");
  configure_apb_agent(m_apb_cfg);
  m_env_cfg.m_apb_agent_cfg = m_apb_cfg;
  // The SPI is not configured as such
  m_spi_cfg.has_functional_coverage = 0;
  m_env_cfg.m_spi_agent_cfg = m_spi_cfg;
  uvm_config_db #(spi_env_config)::set(this, "*", "spi_env_config", m_env_cfg);
  m_env = spi_env::type_id::create("m_env", this);
endfunction: build_phase

function void spi_test_base::set_seqs(spi_vseq_base seq);
  seq.m_cfg = m_env_cfg;
  seq.spi = m_env.m_spi_agent.m_sequencer;
endfunction
```

Assigning Virtual Interfaces From The Configuration Space

Before calling the UVM *run_test()* method, the links to the signals at the top level I/O boundary of the DUT must be made by connecting them to SystemVerilog interfaces to create a pin interface. The pin interface is then connected to the (driver and monitor) BFM interfaces and a handle to each BFM interface is assigned to a virtual interface handle which is passed to the test through a uvm_config_db::set call. See the article on virtual interfaces for more detailed information.

In the *build()* method of the test these virtual interface handles are to be assigned to the virtual interface handles inside the pertinent component configuration object(s). Individual components then access the virtual interface handle inside their configuration object to drive or monitor the DUT via method calls. In order to keep components modular and reusable, drivers and monitors should not retrieve their virtual interface handles directly from the configuration space, but only from their configuration object. The test class is the correct place to ensure that the virtual interfaces are assigned to the right verification components via their configuration objects.

The following code shows the use of the uvm_config_db::get method in the SPI testbench example to make virtual interface assignments to the virtual interface handles in the apb_agent configuration object:

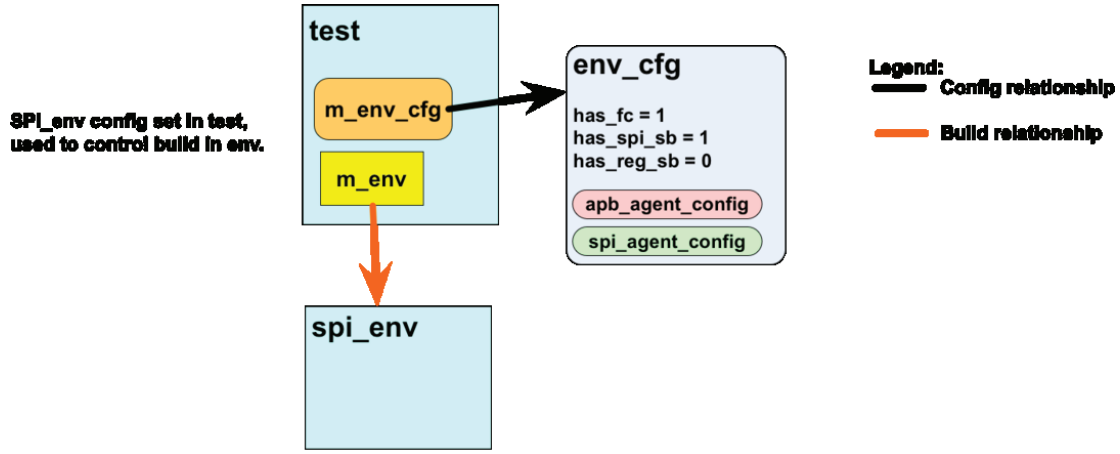
```
// The build method from earlier, adding the apb agent virtual interface assignment
// Build the env, create the env configuration including any sub configurations and
// assign virtual interfaces
function void spi_test_base::build_phase( uvm_phase phase );
  // Create env configuration object
  m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
  // Call function to configure the env
  configure_env(m_env_cfg);
  // Create apb agent configuration object
  m_apb_cfg = apb_agent_config::type_id::create("m_apb_cfg");
  // Call function to configure the apb_agent
  configure_apb_agent(m_apb_cfg);
  // Add the APB driver BFM virtual interface
  if ( !uvm_config_db #(virtual apb_driver_bfm)::get(this, "", "APB_drv_bfm",
  m_apb_cfg.drv_bfm ) ) `uvm_error(...)
  // Add the APB monitor BFM virtual interface
  if ( !uvm_config_db #(virtual apb_monitor_bfm)::get(this, "", "APB_mon_bfm",
  m_apb_cfg.mon_bfm ) ) `uvm_error(...)
   ...
endfunction: build_phase
```

Embedding Sub-Component Configuration Objects

Configuration objects are passed to sub-components via the UVM component configuration space from the test. They can be passed individually, using the path argument in the uvm_config_db::set method to control which components can access the objects. However a common requirement is that intermediate components also need to do some local configuration.

Therefore, an effective way to approach the passing of configuration objects through a testbench hierarchy is to embed the configuration objects inside one another in a way that reflects the hierarchy itself. At each intermediate level in the testbench the configuration object for that level is "unfolded" to yield its sub-configuration objects which are re-configured (if necessary) and then passed to the relevant sub-components using uvm_config_db::set.

Following the SPI block level environment example, each of the agents has a separate configuration object. The env configuration object has a handle to each agent configuration object. In the test, all three configuration objects are constructed and configured from a test case viewpoint, and the agent configuration objects are assigned to the corresponding agent configuration object handles inside the env configuration object. Subsequently the env configuration object is added to the configuration space to be retrieved later when the env is built.



Testbench Build Process – Step 1 – At the end of the test build() method

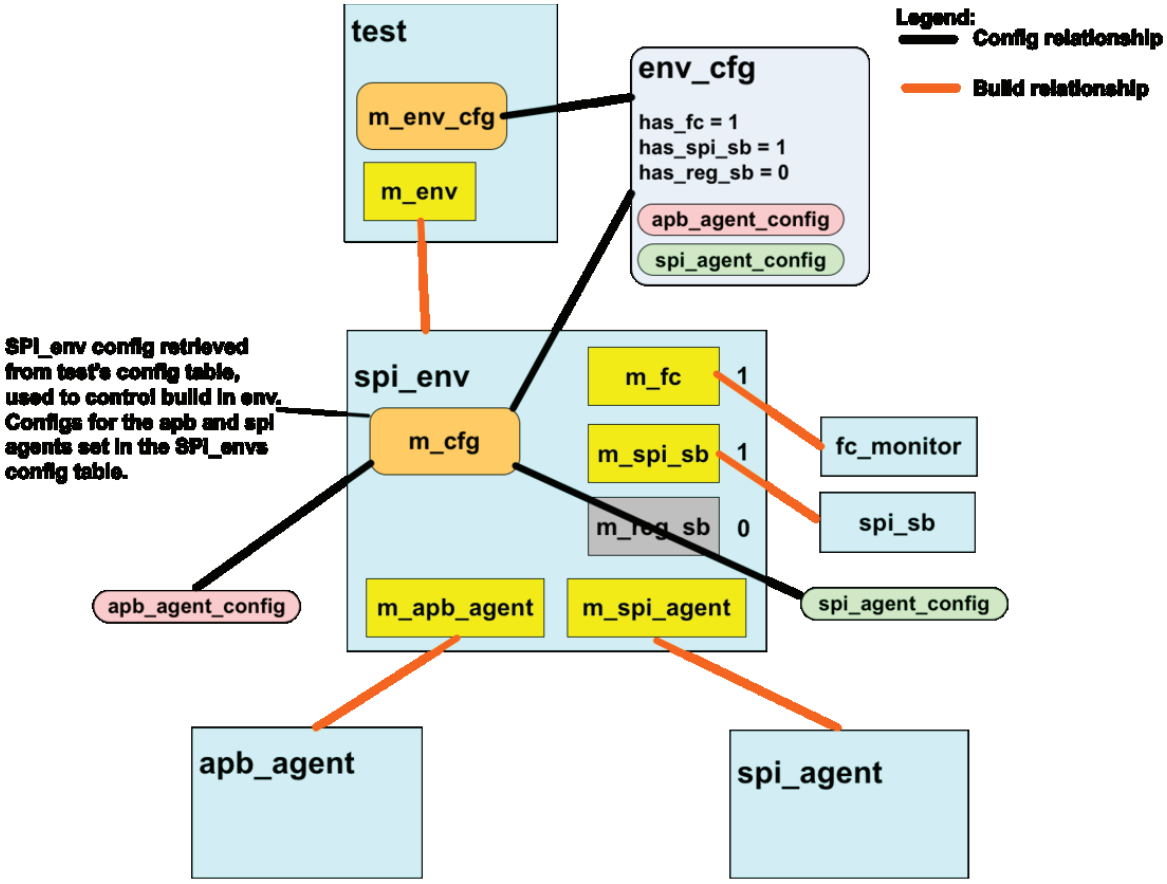
For more complex environments additional levels of encapsulation are required.

```
//
// Configuration object for the spi_env:
//

//
// Class Description:
//
//
class spi_env_config extends uvm_object;

// UVM Factory Registration Macro
//
`uvm_object_utils(spi_env_config)

//------------------------------------------
// Data Members
//------------------------------------------
// Whether env analysis components are used:
  bit has_functional_coverage = 0;
  bit has_spi_functional_coverage = 1;
  bit has_reg_scoreboard = 0;
  bit has_spi_scoreboard = 1;

// Configurations for the sub_components
apb_config m_apb_agent_cfg;
spi_agent_config m_spi_agent_cfg;

//------------------------------------------
// Methods
//------------------------------------------

extern function new(string name = "spi_env_config");
```

```
endclass: spi_env_config

function spi_env_config::new(string name = "spi_env_config");
 super.new(name);
endfunction

//
// Inside the spi_test_base class, the agent config handles are assigned:
//
// The build method from earlier, adding the apb agent virtual interface assignment
// Build the env, create the env configuration including any sub configurations and
// assign virtual interfaces
function void spi_test_base::build_phase( uvm_phase phase );
// Create env configuration object
m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
// Call function to configure the env
configure_env(m_env_cfg);
// Create apb agent configuration object
m_apb_cfg = apb_agent_config::type_id::create("m_apb_cfg");
// Call function to configure the apb_agent
configure_apb_agent(m_apb_cfg);
// Adding the APB monitor BFM virtual interface:
if ( !uvm_config_db #(virtual apb_monitor_bfm)::get(this, "", "APB_mon_bfm",
m_apb_cfg.mon_bfm ) ) `uvm_error(...)
// Adding the APB driver BFM virtual interface:
if ( !uvm_config_db #(virtual apb_driver_bfm)::get(this, "", "APB_drv_bfm",
m_apb_cfg.drv_bfm ) ) `uvm_error(...)
// Assign the apb_agent config handle inside the env_config:
m_env_cfg.m_apb_agent_cfg = m_apb_cfg;
// Repeated for the spi configuration object
m_spi_cfg = spi_agent_config::type_id::create("m_spi_cfg");
configure_spi_agent(m_spi_cfg);
// Adding the SPI driver BFM virtual interface
if ( !uvm_config_db #(virtual spi_driver_bfm)::get(this, "", "SPI_drv_bfm",
m_spi_cfg.drv_bfm ) ) `uvm_error(...)
// Adding the SPI monitor BFM virtual interface
 if ( !uvm_config_db #(virtual spi_monitor_bfm)::get(this, "", "SPI_mon_bfm",
m_spi_cfg.mon_bfm ) ) `uvm_error(...)
m_env_cfg.m_spi_agent_cfg = m_spi_cfg;
// Now env config is complete set it into config space
uvm_config_db #( spi_env_config )::set( this , "*", "spi_env_config",m_env_cfg) );
// Now we are ready to build the spi_env
m_env = spi_env::type_id::create("m_env", this);
endfunction: build_phase
```

Building the Next Level of Hierarchy

The final stage of the test build process is to build the next level of testbench hierarchy using the UVM factory. This usually means building the top level env, but there could be more than one env or there could be a conditional build with a choice between several envs.



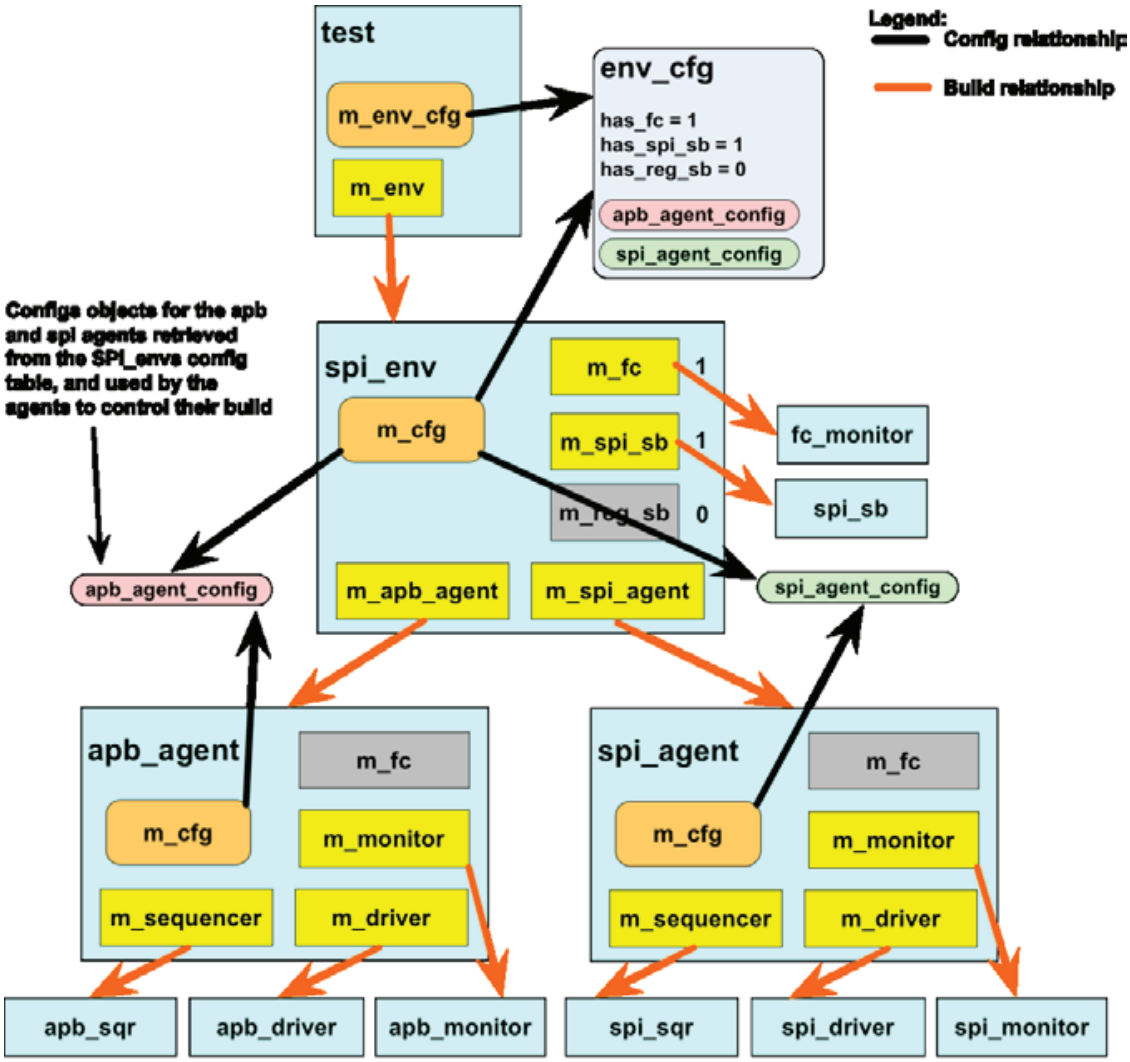
Testbench Build Process – Step 2 – At the end of spi_envs build() method

Coding Convention - Name Argument For Factory Create Method Should Match Local Handle

The *create()* method takes two arguments, one is a name string and the other is a pointer to the parent *uvm_component* class object. The values of these arguments are used to create an entry in a linked list which the UVM uses to locate uvm_components in a pseudo hierarchy. This list is used in the messaging and configuration mechanisms. By convention, the name argument string should be the same as the declaration handle of the component and the parent argument should be the keyword "this" so that it references the uvm_component in which it is created. Using the same name as the handle facilitates cross-referencing paths and handles. For instance, in the previous code snippet, the spi_env is created in the test using its declaration handle m_env, and hence following the build process the UVM "dynamic path" name to this spi_env is "spi_test.m_env".

Hierarchical Build Process

The build phase in the UVM works top-down. Once the *test* class has been constructed, its *build()* method is called followed by the *build()* method of each of its children, and so forth, until the full environment hierarchy has been constructed. This deferred approach to construction enables each *build()* method to affect what happens in the build process of components at lower levels in the hierarchy. For instance, if an agent is configured to be passive, the build process for the agent omits the creation of the agent's sequencer and driver since these are only required if the agent is active.



Testbench Build Process – Stage 3 - End of Agent build() method

The Hierarchical Connection Process

Once the build phase completes, the UVM testbench component hierarchy is in place and the individual components have been constructed and linked into the component hierarchy linked list. The UVM connect phase follows the build phase and works back up from the bottom of the hierarchy to the top. Its purpose is to make TLM connections between components, assign virtual interface handles and make any other assignments for resources such as register models.

Configuration objects are once again at play during the connection process as they may contain references to virtual interfaces or other information that guides the connection process. For instance, inside an agent, the virtual interface assignment to a driver and the TLM connection between a driver and its sequencer are only made if the agent is active.

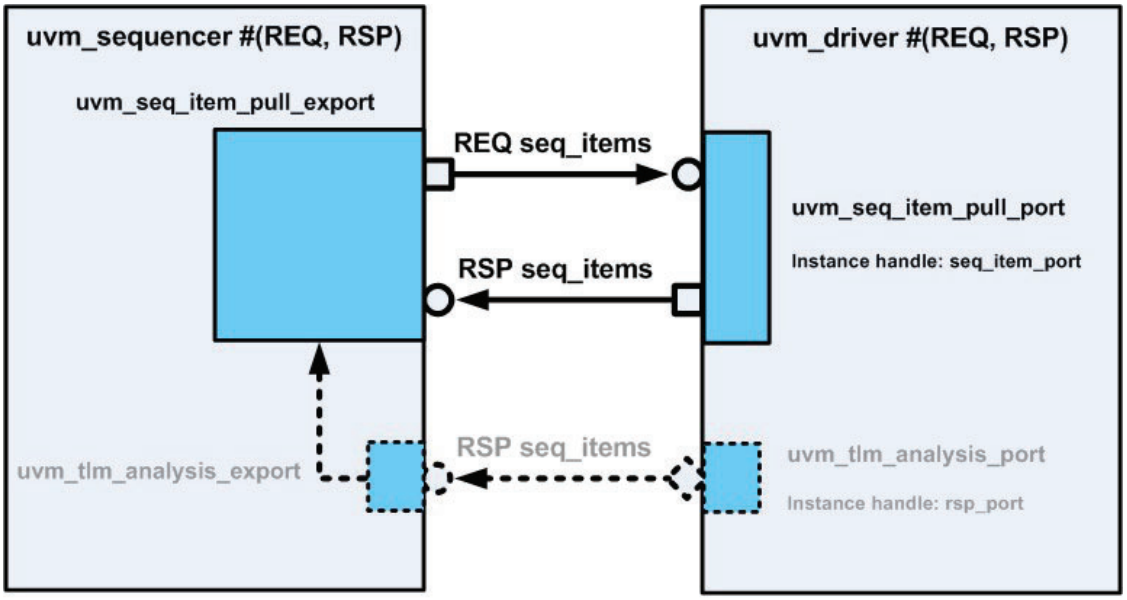
Examples

The UVM build process is best illustrated through some examples that illustrate how different component hierarchies are built up:

- A block level testbench containing an agent
- An integration level testbench

Sequencer-Driver Connections|Connecting the Sequencer and Driver

The transfer of request and response sequence items between sequences and their target driver is facilitated by a bidirectional TLM communication mechanism implemented in the sequencer. The uvm_driver class contains an uvm_seq_item_pull_port which should be connected to an uvm_seq_item_pull_export in the sequencer associated with the driver. The port and export classes are parameterized with the types of the sequence_items that are going to be used for request and response transactions. Once the port-export connection is made, the driver code can use the API implemented in the export to get request sequence_items from sequences and return responses to them.



Sequencer-Driver Connections

The connection between the driver port and the sequencer export is made using a TLM connect method during the connect phase:

```
// Driver parameterized with the same sequence_item for request & response
// response defaults to request
class adpcm_driver extends uvm_driver #(adpcm_seq_item);
....
endclass: adpcm_driver

// Agent containing a driver and a sequencer - uninteresting bits left out
class adpcm_agent extends uvm_agent;

adpcm_driver m_driver;
adpcm_agent_config m_cfg;
// uvm_sequencer parameterized with the adpcm_seq_item for request & response
uvm_sequencer #(adpcm_seq_item) m_sequencer;
```

```
// Sequencer-Driver connection:
function void connect_phase(uvm_phase phase);
  if(m_cfg.active == UVM_ACTIVE) begin
    // The agent is actively driving stimulus
    // Driver-Sequencer TLM connection
    m_driver.seq_item_port.connect(m_sequencer.seq_item_export);
    m_driver.vif = cfg.vif;
    // Virtual interface assignment
  end
endfunction: connect_phase
```

The connection between a driver and a sequencer is typically made in the connect_phase() method of an agent. With the standard UVM driver and sequencer base classes, the TLM connection between a driver and sequencer is a one to one connection - multiple drivers are not connected to a sequencer, nor are multiple sequencers connected to a driver.

In addition to this bidirectional TLM port, there is an analysis_port in the driver which can be connected to an analysis_export in the sequencer to implement a unidirectional response communication path between the driver and the sequencer. This is a historical artifact and provides redundant functionality which is not generally used. The bidirectional TLM interface provides all the functionality required. If this analysis port is used, then the way to connect it is as follows:

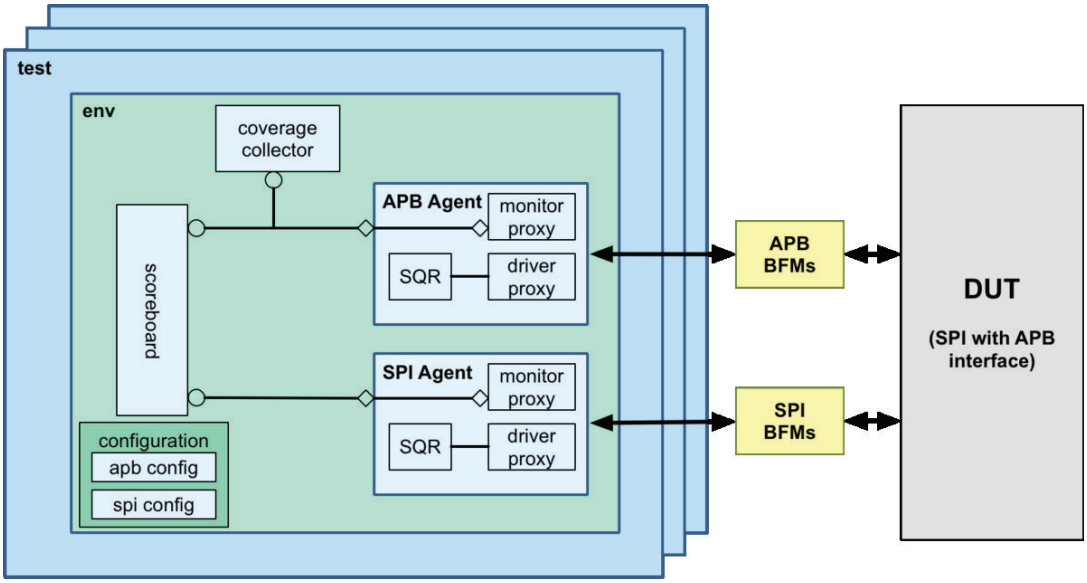
```
// Same agent as in the previous bidirectional example:
class adpcm_agent extends uvm_agent;

adpcm_driver m_driver;
uvm_sequencer #(adpcm_seq_item) m_sequencer;
adpcm_agent_config m_cfg;

// Connect method:
function void connect_phase(uvm_phase phase );
  if(m_cfg.active == UVM_ACTIVE) begin
    // Always need the driver-sequencer TLM connection
    m_driver.seq_item_port.connect(m_sequencer.seq_item_export);

    // Response analysis port connection
    m_driver.rsp_port.connect(m_sequencer.rsp_export);
    m_driver.vif = cfg.vif;
  end
  //...
endfunction: connect_phase

endclass: adpcm_agent
```

Note that the bidirectional TLM connection will always have to be made to effect the communication of requests.

One possible use model for the rsp_port is to notify other components when a driver returns a response, otherwise it is not needed.

Block-Level Testbench

As an example of a block level testbench, consider a testbench built to verify a SPI Master DUT. In this case, the UVM environment has two agents - an APB agent to handle bus transfers on its APB slave port, and a SPI agent to handle SPI protocol transfers on its SPI port. The structure of the overall UVM verification environment is illustrated in the block diagram. Let us go through each layer of the testbench and describe how it is put together from the top down.



The Testbench Modules

Two top level testbench modules are used in the SPI block level testbench. The hdl_top module contains the SPI Master DUT, the APB and SPI BFMs and the apb_if, spi_if and intr_if pin interfaces. The SPI Master DUT is connected to the apb_if, spi_if and intr_if which in turn are connected to the APB and SPI master and slave BFMs, respectively. Two initial blocks are also encapsulated within the hdl_top module. The first initial block places the virtual interface handles for the BFM interfaces into the UVM configuration space using uvm_config_db::set. The second initial block generates a clock and a reset signal for the APB interface.

```
module hdl_top;

`include "timescale.v"

// PCLK and PRESETn
//
logic PCLK;
logic PRESETn;

//
// Instantiate the pin interfaces:
//
apb_if APB(PCLK, PRESETn);
spi_if SPI();
intr_if INTR();
```

```
//
// Instantiate the BFM interfaces:
//
  apb_monitor_bfm APB_mon_bfm(
     .PCLK    (APB.PCLK),
     .PRESETn (APB.PRESETn),
     .PADDR   (APB.PADDR),
     .PRDATA  (APB.PRDATA),
     .PWDATA  (APB.PWDATA),
     .PSEL    (APB.PSEL),
     .PENABLE (APB.PENABLE),
     .PWRITE  (APB.PWRITE),
     .PREADY  (APB.PREADY)
  );
  apb_driver_bfm APB_drv_bfm(
     .PCLK    (APB.PCLK),
     .PRESETn (APB.PRESETn),
     .PADDR   (APB.PADDR),
     .PRDATA  (APB.PRDATA),
     .PWDATA  (APB.PWDATA),
     .PSEL    (APB.PSEL),
     .PENABLE (APB.PENABLE),
     .PWRITE  (APB.PWRITE),
     .PREADY  (APB.PREADY)
  );
  spi_monitor_bfm SPI_mon_bfm(
     .clk  (SPI.clk),
     .cs   (SPI.cs),
     .miso (SPI.miso),
     .mosi (SPI.mosi)
  );
  spi_driver_bfm SPI_drv_bfm(
     .clk  (SPI.clk),
     .cs   (SPI.cs),
     .miso (SPI.miso),
     .mosi (SPI.mosi)
  );
  intr_bfm INTR_bfm(
     .IRQ  (INTR.IRQ),
     .IREQ (INTR.IREQ)
  );
```

```
// DUT
spi_top DUT(
 // APB Interface:
 .PCLK(PCLK),
 .PRESETN(PRESETn),
 .PSEL(APB.PSEL[0]),
 .PADDR(APB.PADDR[4:0]),
 .PWDATA(APB.PWDATA),
 .PRDATA(APB.PRDATA),
 .PENABLE(APB.PENABLE),
 .PREADY(APB.PREADY),
 .PSLVERR(),
 .PWRITE(APB.PWRITE),
 // Interrupt output
 .IRQ(INTR.IRQ),
 // SPI signals
 .ss_pad_o(SPI.cs),
 .sclk_pad_o(SPI.clk),
 .mosi_pad_o(SPI.mosi),
 .miso_pad_i(SPI.miso)
);

// Initial block for virtual interface wrapping:
initial begin
  import uvm_pkg::uvm_config_db;
  uvm_config_db #(virtual apb_monitor_bfm)::set(null, "uvm_test_top", "APB_mon_bfm", APB_mon_bfm);
  uvm_config_db #(virtual apb_driver_bfm) ::set(null, "uvm_test_top", "APB_drv_bfm", APB_drv_bfm);
  uvm_config_db #(virtual spi_monitor_bfm)::set(null, "uvm_test_top", "SPI_mon_bfm", SPI_mon_bfm);
  uvm_config_db #(virtual spi_driver_bfm) ::set(null, "uvm_test_top", "SPI_drv_bfm", SPI_drv_bfm);
  uvm_config_db #(virtual intr_bfm)       ::set(null, "uvm_test_top", "INTR_bfm", INTR_bfm);
end

//
// Initial blocks for clock and reset generation:
//
initial begin
  PCLK = 0;
  forever #10ns PCLK = ~PCLK;
end
initial begin
  PRESETn = 0;
  repeat(4) @(posedge PCLK);
  PRESETn = 1;
end
endmodule: hdl_top
```

The hvl_top module simply imports the uvm_pkg and the spi_test_lib_pkg which contains definitions for the tests that can be run. It also contains the initial block that calls the run_test() method to construct and launch the specified test and thus UVM phasing.

```
module hvl_top;

`include "timescale.v"

import uvm_pkg::*;
import spi_test_lib_pkg::*;

// UVM initial block:
initial begin
  run_test();
end

endmodule: hvl_top
```

The Test

The next phase in the UVM construction process is the build phase. For the SPI block level example this means building the spi_env component, after first creating and preparing all pertinent configuration objects to be used by the environment. The configuration and build process is largely common to most test cases, so it is generally good practice to devise a test base class that can be extended to create specific tests.

In the SPI example, the configuration object for the spi_env contains handles for the SPI and APB configuration objects. This allows the env configuration object to be used to pass all required sub-configuration objects to the env, as part of the build method of the spi_env. This "Russian Doll" approach to nesting configurations is scalable for many levels of hierarchy.

Before the configuration objects for the agents are assigned to their handles in the env configuration block, they are themselves constructed, and their virtual interfaces assigned using the uvm_config_db::get method, and then configured. The virtual interface assignments are to the virtual BFM interface handles that were set in the hdl_top. The APB agent may well be configured differently for different test cases and so its configuration process has been dedicated to a specific virtual method in the base class. This lets derived test classes overload this method and custom configure the APB agent as required.

The following code is for the spi_test_base class:

```
//
// Class Description:
//
//
class spi_test_base extends uvm_test;

// UVM Factory Registration Macro
//
`uvm_component_utils(spi_test_base)
```

```
//----------------------------------------
// Data Members
//----------------------------------------

//----------------------------------------
// Component Members
//----------------------------------------
// The environment class
spi_env m_env;
// Configuration objects
spi_env_config m_env_cfg;
apb_agent_config m_apb_cfg;
spi_agent_config m_spi_cfg;
// Register map
spi_register_map spi_rm;
//Interrupt Utility
intr_util INTR;

//----------------------------------------
// Methods
//----------------------------------------
extern virtual function void configure_apb_agent(apb_agent_config cfg);
// Standard UVM Methods:
extern function new(string name = "spi_test_base", uvm_component parent = null);
extern function void build_phase( uvm_phase phase );

endclass: spi_test_base

function spi_test_base::new(string name = "spi_test_base", uvm_component parent = null);
 super.new(name, parent);
endfunction

// Build the env, create the env configuration
// including any sub configurations and assigning virtual interfaces
function void spi_test_base::build_phase( uvm_phase phase );
 virtual intr_bfm temp_intr_bfm;
 // env configuration
 m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
 // Register map - Keep reg_map a generic name for vertical reuse reasons
 spi_rm = new("reg_map", null);
 m_env_cfg.spi_rm = spi_rm;
 m_apb_cfg = apb_agent_config::type_id::create("m_apb_cfg");
 configure_apb_agent(m_apb_cfg);
 if (!uvm_config_db #(virtual apb_monitor_bfm)::get(this, "", "APB_mon_bfm",
 m_apb_cfg.mon_bfm)) `uvm_fatal(...)
```

```
if (!uvm_config_db #(virtual apb_driver_bfm) ::get(this, "", "APB_drv_bfm",
m_apb_cfg.drv_bfm)) `uvm_fatal(...)
m_spi_cfg.has_functional_coverage = 0;
m_env_cfg.m_spi_agent_cfg = m_spi_cfg;
// Insert the interrupt virtual interface into the env_config:
INTR = intr_util::type_id::create("INTR");
if (!uvm_config_db #(virtual intr_bfm)::get(this, "", "INTR_bfm", temp_intr_bfm) )
`uvm_fatal(...)
INTR.set_bfm(temp_intr_bfm);
m_env_cfg.INTR = INTR;
uvm_config_db #( spi_env_config )::set( this ,"*", "spi_env_config", m_env_cfg);
m_env = spi_env::type_id::create("m_env", this);
// Override for register adapter:
register_adapter_base::type_id::set_inst_override(apb_register_adapter::get_type(),
"spi_bus.adapter");
endfunction: build_phase

//
// Convenience function to configure the apb agent
//
// This can be overloaded by extensions to this base class
function void spi_test_base::configure_apb_agent(apb_agent_config cfg);
 cfg.active = UVM_ACTIVE;
 cfg.has_functional_coverage = 0;
 cfg.has_scoreboard = 0;
 // SPI is on select line 0 for address range 0-18h
 cfg.no_select_lines = 1;
 cfg.start_address[0] = 32'h0;
 cfg.range[0] = 32'h18;
endfunction: configure_apb_agent
```

To create a specific test case, the spi_test_base class is extended, and this allows the test writer to take advantage of the configuration and build process defined in the parent class. As a result, the test writer only needs to add a run_phase method. In the following (simplistic and to be updated) example, the run_phase method instantiates sequences and starts them on appropriate sequencers in the env. All of the configuration process is carried out by the super.build_phase() method call in the build_phase method.

```
//
// Class Description:
//
//
class spi_poll_test extends spi_test_base;
  // UVM Factory Registration Macro
  //
  `uvm_component_utils(spi_poll_test)

  //------------------------------------------
  // Methods
  //------------------------------------------

  // Standard UVM Methods:
  extern function new(string name = "spi_poll_test", uvm_component parent = null);
  extern function void build_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

 endclass: spi_poll_test

 function spi_poll_test::new(string name = "spi_poll_test", uvm_component parent = null);
   super.new(name, parent);
 endfunction

 // Build the env, create the env configuration
 // including any sub configurations and assigning virtual interfaces
 function void spi_poll_test::build_phase(uvm_phase phase);
   super.build_phase(phase);
 endfunction: build_phase

 task spi_poll_test::run_phase(uvm_phase phase);

   config_polling_test t_seq = config_polling_test::type_id::create("t_seq");
   set_seqs(t_seq);

   phase.raise_objection(this, "Test Started");
   t_seq.start(null);
   #100;
   phase.drop_objection(this, "Test Finished");

 endtask: run_phase
```

The Environment

The next level in the SPI UVM environment is the spi_env. This class contains a number of sub-components, namely the SPI and APB agents, a scoreboard and a functional coverage collector. Which of these sub-components gets built is determined by variables in the spi_env configuration object.

In this case, the spi_env configuration object also contains a utility which contains a method for detecting an interrupt. This will be used by ***sequences***. The contents of the spi_env_config class are as follows:

```
//
// Class Description:
//
//
class spi_env_config extends uvm_object;

const string s_my_config_id = "spi_env_config";
const string s_no_config_id = "no config";
const string s_my_config_type_error_id = "config type error";

// UVM Factory Registration Macro
//
`uvm_object_utils(spi_env_config)

// Interrupt Utility - used in the wait for interrupt task
//
intr_util INTR;

//------------------------------------------
// Data Members
//------------------------------------------
// Whether env analysis components are used:
bit has_functional_coverage = 0;
bit has_spi_functional_coverage = 1;
bit has_reg_scoreboard = 0;
bit has_spi_scoreboard = 1;
// Whether the various agents are used:
bit has_apb_agent = 1;
bit has_spi_agent = 1;
// Configurations for the sub_components
apb_agent_config m_apb_agent_cfg;
spi_agent_config m_spi_agent_cfg;
// SPI Register model
uvm_register_map spi_rm;

//------------------------------------------
// Methods
//------------------------------------------
extern task wait_for_interrupt;
extern function bit is_interrupt_cleared;
```

```
// Standard UVM Methods:
extern function new(string name = "spi_env_config");

endclass: spi_env_config

function spi_env_config::new(string name = "spi_env_config");
 super.new(name);
endfunction

// This task is a convenience method for sequences waiting for the interrupt
// signal
task spi_env_config::wait_for_interrupt;
 INTR.wait_for_interrupt();
endtask: wait_for_interrupt

// Check that interrupt has cleared:
function bit spi_env_config::is_interrupt_cleared;
 return INTR.is_interrupt_cleared();
endfunction: is_interrupt_cleared
```

In this example, there are build configuration field bits for each sub-component. This gives the env the ultimate flexibility for reuse.

During the spi_env's build phase, a handle to the spi_env_config is retrieved from the configuration space using uvm_config_db get(). Then the build process tests the various has_<sub_component> fields in the configuration object to determine whether to build a sub-component. In the case of the APB and SPI agents, there is an additional step which is to unpack the configuration objects for each of the agents from the envs configuration object and then to set the agent configuration objects in the envs configuration table after any local modification.

In the connect phase, the spi_env configuration object is again used to determine which TLM connections to make.

```
//
// Class Description:
//
//
class spi_env extends uvm_env;

  // UVM Factory Registration Macro
  //
  `uvm_component_utils(spi_env)
  //------------------------------------------
  // Data Members
  //------------------------------------------
  apb_agent m_apb_agent;
  spi_agent m_spi_agent;
  spi_env_config m_cfg;
  spi_scoreboard m_scoreboard;

  // Register layer adapter
  reg2apb_adapter m_reg2apb;
```

```
  // Register predictor
  uvm_reg_predictor#(apb_seq_item) m_apb2reg_predictor;

  //---------------------------------------
  // Constraints
  //---------------------------------------

  //---------------------------------------
  // Methods
  //---------------------------------------

  // Standard UVM Methods:
  extern function new(string name = "spi_env", uvm_component parent = null);
  extern function void build_phase(uvm_phase phase);
  extern function void connect_phase(uvm_phase phase);
endclass:spi_env

function spi_env::new(string name = "spi_env", uvm_component parent = null);
  super.new(name, parent);
endfunction

function void spi_env::build_phase(uvm_phase phase);
  if (!uvm_config_db #(spi_env_config)::get(this, "", "spi_env_config", m_cfg))
    `uvm_fatal("CONFIG_LOAD", "Cannot get() configuration spi_env_config from
    uvm_config_db. Have you set() it?")
  uvm_config_db #(apb_agent_config)::set(this, "m_apb_agent*",
    "apb_agent_config",
    m_cfg.m_apb_agent_cfg);
  m_apb_agent = apb_agent::type_id::create("m_apb_agent", this);

  // Build the register model predictor
  m_apb2reg_predictor =
  uvm_reg_predictor#(apb_seq_item)::type_id::create("m_apb2reg_predictor", this);
  m_reg2apb = reg2apb_adapter::type_id::create("m_reg2apb");
  uvm_config_db #(spi_agent_config)::set(this, "m_spi_agent*",
    "spi_agent_config",
    m_cfg.m_spi_agent_cfg);
  m_spi_agent = spi_agent::type_id::create("m_spi_agent", this);

  if(m_cfg.has_spi_scoreboard) begin
    m_scoreboard = spi_scoreboard::type_id::create("m_scoreboard", this);
  end
endfunction:build_phase
```

```
function void spi_env::connect_phase(uvm_phase phase);

  // Only set up register sequencer layering if the spi_rb is the top block
  // If it isn't, then the top level environment will set up the correct sequencer
  // and predictor
  if(m_cfg.spi_rb.get_parent() == null) begin
   if(m_cfg.m_apb_agent_cfg.active == UVM_ACTIVE) begin
   m_cfg.spi_rb.spi_reg_block_map.set_sequencer(m_apb_agent.m_sequencer, m_reg2apb);
    end

    //
    // Register prediction part:
    //
    // Replacing implicit register model prediction with explicit prediction
    // based on APB bus activity observed by the APB agent monitor
    // Set the predictor map:
    m_apb2reg_predictor.map = m_cfg.spi_rb.spi_reg_block_map;
    // Set the predictor adapter:
    m_apb2reg_predictor.adapter = m_reg2apb;
    // Disable the register models auto-prediction
    m_cfg.spi_rb.spi_reg_block_map.set_auto_predict(0);
    // Connect the predictor to the bus agent monitor analysis port
    m_apb_agent.ap.connect(m_apb2reg_predictor.bus_in);
  end

  if(m_cfg.has_spi_scoreboard) begin
    m_spi_agent.ap.connect(m_scoreboard.spi.analysis_export);
    m_scoreboard.spi_rb = m_cfg.spi_rb;
  end
endfunction: connect_phase
```

The Agents

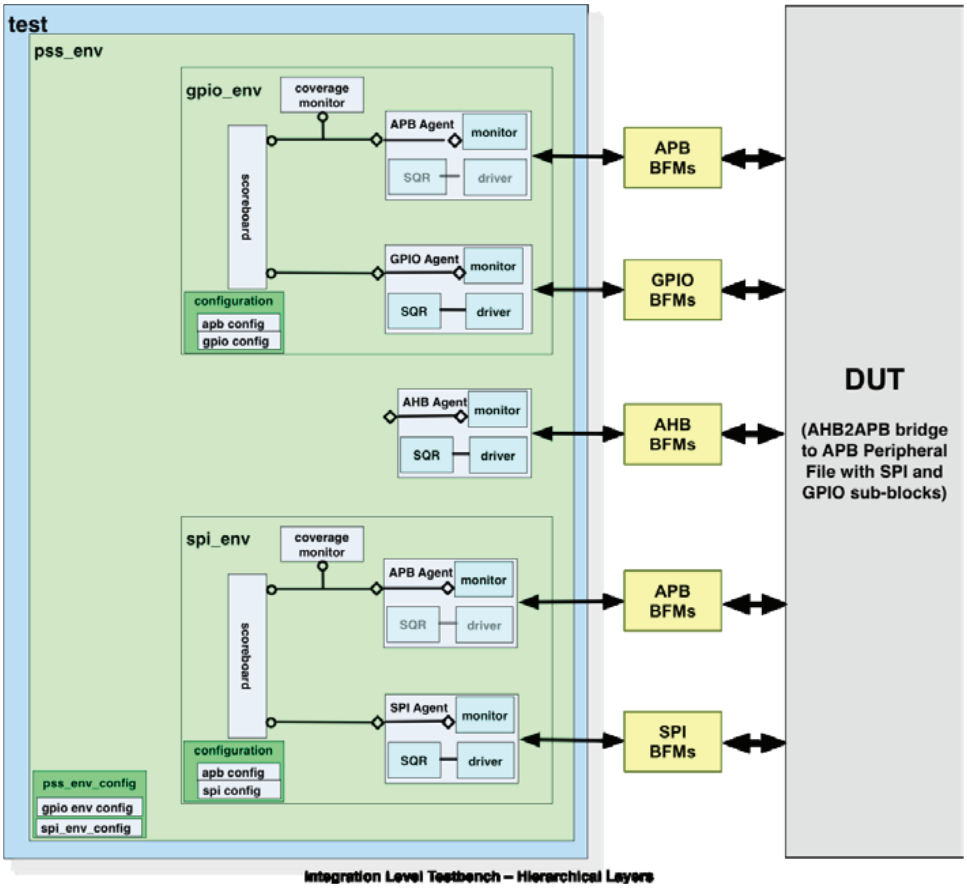
Since the UVM build process is top down, the SPI and APB agents are constructed next. The article on the ***agent build process*** describes how the APB agent is configured and built, and the SPI agent follows the same process.

The components within the agents are at the bottom of the testbench hierarchy, so the build process terminates there.

Integration-Level Testbench

This testbench example is one that takes two block level verification environments and shows how they can be reused at a higher level of integration. The principles that are illustrated in the example are applicable to repeated rounds of vertical reuse.

The example takes the SPI block level example and integrates it with another block level verification environment for a GPIO DUT. The hardware for the two blocks has been integrated into a Peripheral Sub-System (PSS) which uses an AHB to APB bus bridge to interface with the APB interfaces on the SPI and GPIO blocks. The environments from the block level are encapsulated by the pss_env, which also includes an AHB agent to drive the exposed AHB bus interface. In this configuration, the block level APB bus interfaces are no longer exposed, and so the APB agents are put into passive mode to monitor the APB traffic. The stimulus needs to drive the AHB interface and register layering enables reuse of block level stimulus at the integration level.



Integration Level Testbench – Hierarchical Layers

We shall now go through the testbench and the build process from the top down, starting with the two top level testbench modules.

Top Level Testbench Modules

As with the block level testbench example, two top level modules are utilized. The hdl_top instantiates the DUT, instantiates the BFM interfaces and connects the pin interfaces to the DUT and the BFM interfaces. Virtual Interface handles referencing the BFM interfaces are placed into the configuration space and clocks and resets are generated. The main differences between this code and the block level testbench code are that there are more interfaces and that there is a need to bind to some internal signals to monitor the APB bus. Another differences is that driver BFMs are not instantiated if the agent is going to be used in passive mode. The DUT is wrapped by a module which connects its I/O signals to the interfaces used in the UVM testbench. The internal signals are bound to the APB interface using the binder module:

```
module top_tb;

import uvm_pkg::*;
import pss_test_lib_pkg::*;

// PCLK and PRESETn
//
logic HCLK;
logic HRESETn;

//
// Instantiate the pin interfaces:
//
apb_if APB(HCLK, HRESETn);
// APB interface - shared between passive
agents
ahb_if AHB(HCLK, HRESETn);
// AHB interface
spi_if SPI();
// SPI Interface
...
// Additional pin interfaces

//
// Instantiate the BFM interfaces:
//
apb_monitor_bfm APB_SPI_mon_bfm(

    .PCLK    (APB.PCLK),
    .PRESETn (APB.PRESETn),
    .PADDR    (APB.PADDR),
    .PRDATA   (APB.PRDATA),
    .PWDATA   (APB.PWDATA),
    .PSEL     (APB.PSEL),
    .PENABLE (APB.PENABLE),
    .PWRITE   (APB.PWRITE),
    .PREADY   (APB.PREADY)
);
apb_monitor_bfm APB_GPIO_mon_bfm(
    .PCLK    (APB.PCLK),
    .PRESETn (APB.PRESETn),
    .PADDR    (APB.PADDR),
    .PRDATA  (APB.PRDATA),
```

```
            .PWDATA    (APB.PWDATA),
            .PSEL      (APB.PSEL),
            .PENABLE   (APB.PENABLE),
            .PWRITE    (APB.PWRITE),
            .PREADY    (APB.PREADY)
        );
          apb_driver_bfm APB_GPIO_drv_bfm(
            .PCLK     (APB_dummy.PCLK),
            .PRESETn  (APB_dummy.PRESETn),
            .PADDR      (APB_dummy.PADDR),
            .PRDATA     (APB_dummy.PRDATA),
            .PWDATA     (APB_dummy.PWDATA),
            .PSEL       (APB_dummy.PSEL),
            .PENABLE    (APB_dummy.PENABLE),
            .PWRITE     (APB_dummy.PWRITE),
            .PREADY     (APB_dummy.PREADY)
        );
        ...
        // Additional BFM interfaces
        // Binder
        binder probe();

        // DUT Wrapper:
        pss_wrapper wrapper(.ahb(AHB),
                            .spi(SPI),
                            .gpi(GPI),
                            .gpo(GPO),
                            .gpoe(GPOE),
                            .icpit(ICPIT),
                            .uart_rx(UART_RX),
                            .uart_tx(UART_TX),
                            .modem(MODEM));
        // UVM initial block:
        // Virtual interface wrapping
        initial begin
           import uvm_pkg::uvm_config_db;
uvm_config_db  #(virtual apb_monitor_bfm)     ::set(null, "uvm_test_top", "APB_SPI_mon_bfm", APB_SPI_mon_bfm);
uvm_config_db  #(virtual apb_monitor_bfm)     ::set((null, "uvm_test_top", "APB_GPIO_mon_bfm", APB_GPIO_mon_bfm);
uvm_config_db #(virtual ahb_monitor_bfm) ::set(null, "uvm_test_top", "AHB_mon_bfm", AHB_mon_bfm);
uvm_config_db #(virtual ahb_driver_bfm)  ::set(null, "uvm_test_top", "AHB_drv_bfm", AHB_drv_bfm);
uvm_config_db #(virtual spi_monitor_bfm) ::set(null, "uvm_test_top", "SPI_mon_bfm", SPI_mon_bfm);
uvm_config_db #(virtual spi_driver_bfm)  ::set(null, "uvm_test_top", "SPI_drv_bfm", SPI_drv_bfm);
...
//Additional uvm_config_db::set() calls
end
```

```
//
// Clock and reset initial block:
//
initial begin
   HCLK = 1;
   forever #10ns HCLK = ~HCLK;
end
initial begin
   HRESETn = 0;
   repeat(4) @(posedge HCLK);
   HRESETn = 1;
end

// Clock assignments:
assign GPO.clk = HCLK;
assign GPOE.clk = HCLK;
assign GPI.clk = HCLK;

endmodule: hdl_top
```

The hvl_top module remains largely the same as in the block level example. It now imports the pss_test_lib_pkg so that the definition of the tests are known. Otherwise, the same functionality is present.

```
module hvl_top;

import uvm_pkg::*;
import pss_test_lib_pkg::*;

// UVM initial block:
initial begin
  run_test();
end

endmodule: hvl_top
```

The Test

Like the block level test, the integration level test should have the common build and configuration process captured in a base class that subsequent test cases can inherit from. As can be seen from the example, there is more configuration to do and so the need becomes more compelling.

The configuration object for the pss_env contains handles for the configuration objects for the spi_env and the gpio_env. In turn, the sub-env configuration objects contain handles for their agent sub-component configuration objects. The pss_env is responsible for un-nesting the spi_env and gpio_env configuration objects and setting them in its configuration table, making any local changes necessary. In turn, the spi_env and the gpio_env put their agent configurations into their configuration table.

The pss test base class is as follows:

```
//
// Class Description:
//
//
class pss_test_base extends uvm_test;

// UVM Factory Registration Macro
//
`uvm_component_utils(pss_test_base)

//------------------------------------------
// Data Members
//------------------------------------------

//------------------------------------------
// Component Members
//------------------------------------------
// The environment class
pss_env m_env;
// Configuration objects
pss_env_config m_env_cfg;
spi_env_config m_spi_env_cfg;
gpio_env_config m_gpio_env_cfg;
apb_agent_config m_spi_apb_agent_cfg;
apb_agent_config m_gpio_apb_agent_cfg;
ahb_agent_config m_ahb_agent_cfg;
spi_agent_config m_spi_agent_cfg;
...
// Additional configuration object handles

// Register map
pss_register_map pss_rm;

// Interrupt Utility
intr_util ICPIT;

//------------------------------------------
```

```
// Methods
//-----------------------------------------
// Standard UVM Methods:
extern function new(string name = "spi_test_base", uvm_component parent = null);
extern function void build_phase( uvm_phase phase);
extern virtual function void configure_apb_agent(apb_agent_config cfg, int index,
logic[31:0] start_address, logic[31:0] range);

extern task run_phase( uvm_phase phase );

endclass: pss_test_base

function pss_test_base::new(string name = "spi_test_base", uvm_component parent = null);
  super.new(name, parent);
endfunction

// Build the env, create the env configuration
// including any sub configurations and assigning virtual interfaces
function void pss_test_base::build_phase(uvm_phase phase);
  virtual intr_bfm temp_intr_bfm;

  m_env_cfg = pss_env_config::type_id::create("m_env_cfg");

  // Register model
  // Enable all types of coverage available in the register model
  uvm_reg::include_coverage("*", UVM_CVR_ALL);
  // Register map - Keep reg_map a generic name for vertical reuse reasons
  pss_rb = pss_reg_block::type_id::create("pss_rb");
  pss_rb.build();
  m_env_cfg.pss_rb = pss_rb;

  // SPI Sub-env configuration:
  m_spi_env_cfg = spi_env_config::type_id::create("m_spi_env_cfg");
  m_spi_env_cfg.spi_rb = pss_rb.spi_rb;

  // apb agent in the SPI env:

  m_spi_apb_agent_cfg = apb_agent_config::type_id::create("m_spi_apb_agent_cfg");
  configure_apb_agent(m_spi_apb_agent_cfg, 0, 32'h0, 32'h18);
  if (!uvm_config_db #(virtual apb_monitor_bfm)::get(this, "", "APB_SPI_mon_bfm",
  m_spi_apb_agent_cfg.mon_bfm))
    `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface APB_SPI_mon_bfm from
  uvm_config_db. Have you set() it?")
```

```
// if (!uvm_config_db #(virtual apb_driver_bfm) ::get(this, "", "APB_SPI_drv_bfm",
m_spi_apb_agent_cfg.drv_bfm))
// `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface APB_SPI_drv_bfm from
uvm_config_db. Have you set() it?")
m_spi_apb_agent_cfg.active = UVM_PASSIVE;
m_spi_env_cfg.m_apb_agent_cfg = m_spi_apb_agent_cfg;

// SPI agent:
m_spi_agent_cfg = spi_agent_config::type_id::create("m_spi_agent_cfg");
if (!uvm_config_db #(virtual spi_monitor_bfm)::get(this, "", "SPI_mon_bfm",
m_spi_agent_cfg.mon_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface SPI_mon_bfm from uvm_config_db.
Have you set() it?")
  if (!uvm_config_db #(virtual spi_driver_bfm) ::get(this, "", "SPI_drv_bfm",
m_spi_agent_cfg.drv_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface SPI_drv_bfm from
uvm_config_db. Have you set() it?")
m_spi_env_cfg.m_spi_agent_cfg = m_spi_agent_cfg;
m_env_cfg.m_spi_env_cfg = m_spi_env_cfg;
uvm_config_db #(spi_env_config)::set(this, "*", "spi_env_config", m_spi_env_cfg);

// GPIO env configuration:
m_gpio_env_cfg = gpio_env_config::type_id::create("m_gpio_env_cfg");
m_gpio_env_cfg.gpio_rb = pss_rb.gpio_rb;
m_gpio_apb_agent_cfg = apb_agent_config::type_id::create("m_gpio_apb_agent_cfg");
configure_apb_agent(m_gpio_apb_agent_cfg, 1, 32'h100, 32'h124);
if (!uvm_config_db #(virtual apb_monitor_bfm)::get(this, "", "APB_GPIO_mon_bfm",
m_gpio_apb_agent_cfg.mon_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface APB_GPIO_mon_bfm
from uvm_config_db. Have you set() it?")
// if (!uvm_config_db #(virtual apb_driver_bfm) ::get(this, "", "APB_GPIO_drv_bfm",
m_gpio_apb_agent_cfg.drv_bfm))
// `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface APB_drv_bfm from
uvm_config_db. Have you set() it?")
m_gpio_apb_agent_cfg.active = UVM_PASSIVE;
m_gpio_env_cfg.m_apb_agent_cfg = m_gpio_apb_agent_cfg;
m_gpio_env_cfg.has_functional_coverage = 1;
// Register coverage no longer valid

// GPO agent
m_GPO_agent_cfg = gpio_agent_config::type_id::create("m_GPO_agent_cfg");
if (!uvm_config_db #(virtual gpio_monitor_bfm)::get(this, "", "GPO_mon_bfm",
m_GPO_agent_cfg.mon_bfm))
```

```
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface GPO_mon_bfm from
uvm_config_db. Have you set() it?")
// if (!uvm_config_db #(virtual gpio_driver_bfm) ::get(this, "",
"GPO_drv_bfm", m_GPO_agent_cfg.drv_bfm))
// `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface SPI_drv_bfm from
uvm_config_db. Have you set() it?")
m_GPO_agent_cfg.active = UVM_PASSIVE;
// Only monitors
m_gpio_env_cfg.m_GPO_agent_cfg = m_GPO_agent_cfg;

// GPOE agent
m_GPOE_agent_cfg = gpio_agent_config::type_id::create("m_GPOE_agent_cfg");
if (!uvm_config_db #(virtual gpio_monitor_bfm)::get(this, "", "GPOE_mon_bfm",
m_GPOE_agent_cfg.mon_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface GPOE_mon_bfm from
uvm_config_db. Have you set() it?")
// if (!uvm_config_db #(virtual gpio_driver_bfm) ::get(this, "",
"GPOE_drv_bfm", m_GPOE_agent_cfg.drv_bfm))
// `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface SPI_drv_bfm
from uvm_config_db. Have you set() it?")
m_GPOE_agent_cfg.active = UVM_PASSIVE;
// Only monitors
m_gpio_env_cfg.m_GPOE_agent_cfg = m_GPOE_agent_cfg;

// GPI agent - active (default)
m_GPI_agent_cfg = gpio_agent_config::type_id::create("m_GPI_agent_cfg");
if (!uvm_config_db #(virtual gpio_monitor_bfm)::get(this, "", "GPI_mon_bfm",
m_GPI_agent_cfg.mon_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface GPI_mon_bfm from
uvm_config_db. Have you set() it?")
if (!uvm_config_db #(virtual gpio_driver_bfm) ::get(this, "", "GPI_drv_bfm",
m_GPI_agent_cfg.drv_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface SPI_drv_bfm from
uvm_config_db. Have you set() it?")
m_gpio_env_cfg.m_GPI_agent_cfg = m_GPI_agent_cfg;

// GPIO Aux agent not present
m_gpio_env_cfg.has_AUX_agent = 0;
m_gpio_env_cfg.has_functional_coverage = 1;
m_gpio_env_cfg.has_out_scoreboard = 1;
m_gpio_env_cfg.has_in_scoreboard = 1;
m_env_cfg.m_gpio_env_cfg = m_gpio_env_cfg;
uvm_config_db #(gpio_env_config)::set(this, "*", "gpio_env_config", m_gpio_env_cfg);
```

```
// AHB Agent
m_ahb_agent_cfg = ahb_agent_config::type_id::create("m_ahb_agent_cfg");
if (!uvm_config_db #(virtual ahb_monitor_bfm)::get(this, "", "AHB_mon_bfm",
m_ahb_agent_cfg.mon_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface AHB_mon_bfm from
uvm_config_db. Have you set() it?")
if (!uvm_config_db #(virtual ahb_driver_bfm) ::get(this, "", "AHB_drv_bfm",
m_ahb_agent_cfg.drv_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() BFM interface AHB_drv_bfm from
uvm_config_db. Have you set() it?")
m_env_cfg.m_ahb_agent_cfg = m_ahb_agent_cfg;

// Add in interrupt line
ICPIT = intr_util::type_id::create("ICPIT");
if (!uvm_config_db #(virtual intr_bfm)::get(this, "", "ICPIT_bfm", temp_intr_bfm))
  `uvm_fatal("VIF CONFIG", "Cannot get() interface ICPIT_bfm from uvm_config_db.
Have you set() it?")
ICPIT.set_bfm(temp_intr_bfm);
m_env_cfg.ICPIT = ICPIT;
m_spi_env_cfg.INTR = ICPIT;

uvm_config_db #(pss_env_config)::set(this, "*", "pss_env_config", m_env_cfg);
m_env = pss_env::type_id::create("m_env", this);
endfunction: build_phase

//
// Convenience function to configure the apb agent
//
// This can be overloaded by extensions to this base class
function void pss_test_base::configure_apb_agent(apb_agent_config cfg, int index,
logic[31:0] start_address, logic[31:0] range);
   cfg.active = UVM_PASSIVE;
   cfg.has_functional_coverage = 0;
   cfg.has_scoreboard = 0;
   cfg.no_select_lines = 1;
   cfg.apb_index = index;
   cfg.start_address[0] = start_address;
   cfg.range[0] = range;
endfunction: configure_apb_agent

task pss_test_base::run_phase( uvm_phase phase );

endtask: run_phase
```

Again, a test case that extends this base class would populate its run method to define a virtual sequence that would be run on the virtual sequencer in the env. If there is non-default configuration to be done, then this could be done bypopulating or overloading the build method or any of the configuration methods.

```
//
// Class Description:
//
//
class pss_spi_polling_test extends pss_test_base;

  // UVM Factory Registration Macro
  //
  `uvm_component_utils(pss_spi_polling_test)

  //------------------------------------------
  // Methods
  //------------------------------------------

  // Standard UVM Methods:
  extern function new(string name = "pss_spi_polling_test", uvm_component parent = null);
  extern function void build_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);
endclass: pss_spi_polling_test

function pss_spi_polling_test::new(string name = "pss_spi_polling_test",
uvm_component parent = null);
  super.new(name, parent);
endfunction

// Build the env, create the env configuration
// including any sub configurations and assigning virtual interfaces
function void pss_spi_polling_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
endfunction: build_phase

task pss_spi_polling_test::run_phase(uvm_phase phase);
  config_polling_test t_seq = config_polling_test::type_id::create("t_seq");

   t_seq.m_cfg = m_spi_env_cfg;
   t_seq.spi = m_env.m_spi_env.m_spi_agent.m_sequencer;

   phase.raise_objection(this, "Starting PSS SPI polling test");

   repeat(10) begin
     t_seq.start(null);
   end

   phase.drop_objection(this, "Finishing PSS SPI polling test");
   endtask: run_phase
```

The PSS env

The PSS env build process retrieves the configuration object and constructs the various sub-envs, after testing the various has_<sub-component> fields in order to determine whether the env is required by the test case. If the sub-env is to be present, the sub-envs configuration object is set in the PSS envs configuration table. The connect method is used to make connections between TLM ports and exports between monitors and analysis components such as scoreboards.

```
//
// Class Description:
//
//
class pss_env extends uvm_env;

  // UVM Factory Registration Macro
  //
  `uvm_component_utils(pss_env)

  //------------------------------------------
  // Data Members
  //------------------------------------------
  pss_env_config m_cfg;
  //------------------------------------------
  // Sub Components
  //------------------------------------------
  spi_env m_spi_env; gpio_env
  m_gpio_env; ahb_agent
  m_ahb_agent;

  // Register layer adapter
  reg2ahb_adapter m_reg2ahb;
  // Register predictor
  uvm_reg_predictor#(ahb_seq_item) m_ahb2reg_predictor;

  //------------------------------------------
  // Methods
  //------------------------------------------

  // Standard UVM Methods:
  extern function new(string name = "pss_env", uvm_component parent = null);
  // Only required if you have sub-components
  extern function void build_phase(uvm_phase phase);
  // Only required if you have sub-components which are connected
  extern function void connect_phase(uvm_phase phase);

endclass: pss_env
```

```
function pss_env::new(string name = "pss_env", uvm_component parent = null);
  super.new(name, parent);
endfunction

// Only required if you have sub-components
function void pss_env::build_phase(uvm_phase phase);
  if (!uvm_config_db #(pss_env_config)::get(this, "", "pss_env_config", m_cfg) )
    `uvm_fatal("CONFIG_LOAD", "Cannot get() configuration pss_env_config from
  uvm_config_db. Have you set() it?")

  uvm_config_db #(spi_env_config)::set(this, "m_spi_env*", "spi_env_config",
  m_cfg.m_spi_env_cfg); m_spi_env = spi_env::type_id::create("m_spi_env", this);

  uvm_config_db #(gpio_env_config)::set(this, "m_gpio_env*", "gpio_env_config",
  m_cfg.m_gpio_env_cfg); m_gpio_env = gpio_env::type_id::create("m_gpio_env",
  this);

  uvm_config_db #(ahb_agent_config)::set(this, "m_ahb_agent*", "ahb_agent_config",
  m_cfg.m_ahb_agent_cfg); m_ahb_agent = ahb_agent::type_id::create("m_ahb_agent",
  this);

  // Build the register model predictor
  m_ahb2reg_predictor = uvm_reg_predictor#(ahb_seq_item)::type_id::create
  ("m_ahb2reg_predictor", this); m_reg2ahb =
  reg2ahb_adapter::type_id::create("m_reg2ahb");
endfunction: build_phase

// Only required if you have sub-components which are connected
function void pss_env::connect_phase(uvm_phase phase);
  // Only set up register sequencer layering if the pss_rb is the top block
  // If it isn't, then the top level environment will set up the correct sequencer
  // and predictor
  if(m_cfg.pss_rb.get_parent() == null) begin
    if(m_cfg.m_ahb_agent_cfg.active == UVM_ACTIVE) begin
      m_cfg.pss_rb.pss_map.set_sequencer(m_ahb_agent.m_sequencer, m_reg2ahb);
    end
```

```
    //
    // Register prediction part:
    //
    // Replacing implicit register model prediction with explicit prediction
    // based on APB bus activity observed by the APB agent monitor
    // Set the predictor map:
    m_ahb2reg_predictor.map = m_cfg.pss_rb.pss_map;
    // Set the predictor adapter:
    m_ahb2reg_predictor.adapter = m_reg2ahb;
    // Disable the register models auto-prediction
    m_cfg.pss_rb.pss_map.set_auto_predict(0);
    // Connect the predictor to the bus agent monitor analysis port
    m_ahb_agent.ap.connect(m_ahb2reg_predictor.bus_in);

  end
endfunction: connect_phase
```

The rest of the testbench hierarchy

The build process continues top-down with the sub-envs being conditionally constructed as illustrated in the block level testbench example and the agents contained within the sub-envs being built as described in the agent example.

Further levels of integration

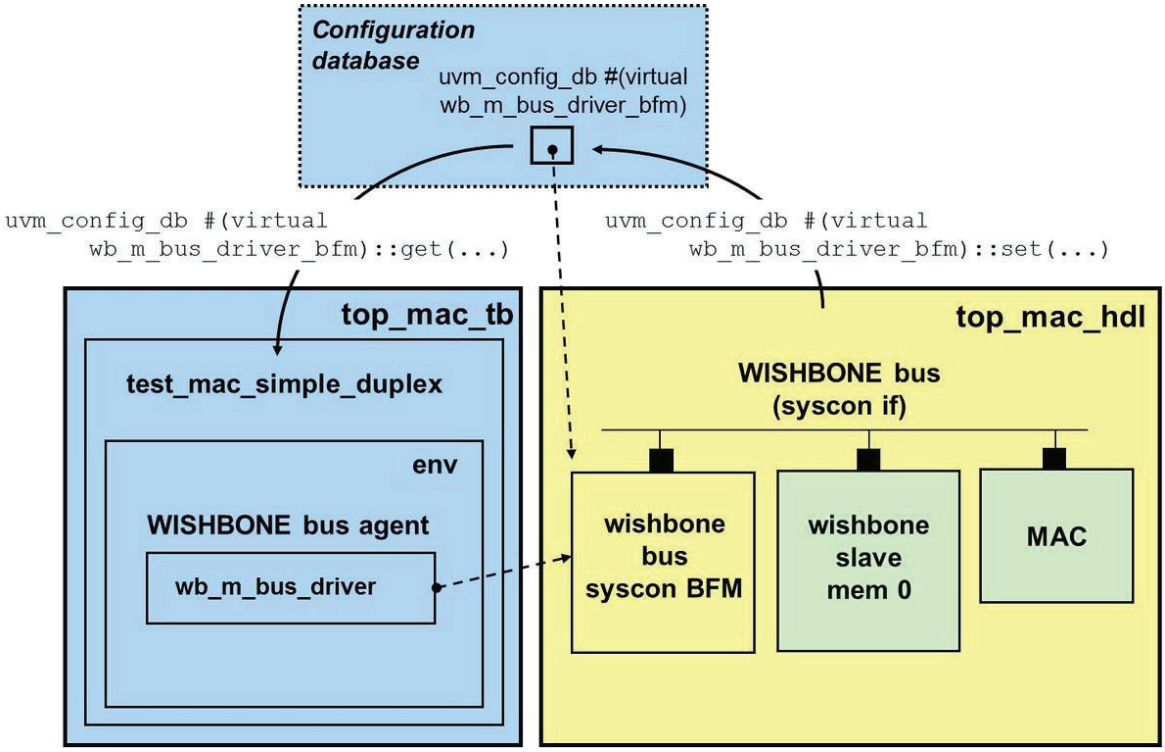
Vertical reuse for further levels of integration can be achieved by extending the process described for the PSS example. Each level of integration adds another layer, so for instance a level 2 integration environment would contain two or more level 1 envs and the level 2 env configuration object would contain nested handles for the level 1 env configuration objects. Obviously, at the test level of the hierarchy the amount of code increases for each round of vertical reuse, but further down the hierarchy, the configuration and build process has already been implemented in the previous generation of vertical layering.

Dual Top Architecture

The dual top testbench architecture advocated throughout this cookbook enables platform portability - it is fundamental for testbench acceleration using emulation or some other hardware-assisted platform. The HDL top level module encapsulates everything associated directly with the clock cycle-based signal level activity of the RTL DUT, which can be run in simulation or be mapped (i.e. synthesized) onto the emulator. The HVL/TB top level module encapsulates the object-oriented testbench and always runs on the simulator as per usual.

Next to hardware-assisted testbench acceleration there are certainly other good reasons to employ a dual top testbench structure. Specifically, it can facilitate the use of multi-processor platforms for simulation, the use of compile and run-time optimization techniques, or the application of good software engineering practices for the creation of portable, configurable VIP. Having more than one top level module in a simulation complies with the (System)Verilog standard and is quite common indeed. All top level (i.e. uninstantiated) modules are effectively treated as implicit instances at the top of the module hierarchy.

Clearly, the dual HDL and testbench top level module hierarchies must be interconnected, or "bound" together to enable TB - HDL inter-domain communication. This is facilitated using the UVM configuration database as depicted below.



HDL top level module

For the MAC example, the *top_mac_hdl* module contains the MAC DUT along with its associated (signal and BFM) interfaces, a MAC MII wrapper module, and the WISHBONE slave memory with WISHBONE bus logic:

```
module top_mac_hdl;
  import test_params_pkg::*;

  // WISHBONE interface instance
  // Supports up to 8 masters and up to 8 slaves
  wishbone_bus_syscon_if wb_bus_if();
```

```
  // BFM interface instances
  wb_m_bus_driver_bfm wb_drv_bfm(wb_bus_if);
  wb_bus_monitor_bfm  wb_mon_bfm(wb_bus_if);

  //-----------------------------------
  //  WISHBONE 0, slave 0:  000000 - 0fffff
  //  this is 1 Mbytes of memory
  wb_slave_mem #(MEM_SLAVE_SIZE) wb_s_0 (
    ...
  );

  ...

  //-----------------------------------
  // MAC 0
  // It is WISHBONE slave 1: address range 100000 - 100fff
  // It is WISHBONE Master 0
  eth_top mac_0 (
    ...
  );

  // Wrapper module for MAC MII interface
  mac_mii_protocol_module #(.INTERFACE_NAME("MIIM_IF")) mii_pm(
    ...
  );

  initial begin
    // Set BFM interfaces in config space
    import uvm_pkg::uvm_config_db;
    uvm_config_db #(virtual wb_m_bus_driver_bfm)::
      set(null, "uvm_test_top", "WB_DRV_BFM", wb_drv_bfm);
    uvm_config_db #(virtual wb_bus_monitor_bfm)::
      set(null, "uvm_test_top", "WB_MON_BFM", wb_mon_bfm);
  end
endmodule
```

Note the use of the explicitly named import of uvm_config_db local to the initial block to put the BFM interfaces in the UVM config space, which is appropriately discriminate in contrast to the test parameters package wildcard import at the beginning of the *top_mac_hdl* top module. The indiscriminate wild card import on the other hand is appropriate for the test parameters package because multiple elements of that package will generally be used inside *top_mac_hdl*.

Further note that instead of placing the uvm_config_db registration of the BFM interfaces under the HDL top level module, as shown above, this may similarly be placed under the HVL top level module using full cross-domain hierarchical instance paths.

```
initial begin
  // Set BFM interfaces in config space
```

```
  import uvm_pkg::uvm_config_db;
  uvm_config_db #(virtual wb_m_bus_driver_bfm)::
    set(null, "uvm_test_top", "WB_DRV_BFM", top_mac_hdl.wb_drv_bfm);
  uvm_config_db #(virtual wb_bus_monitor_bfm)::
    set(null, "uvm_test_top", "WB_MON_BFM", top_mac_hdl.wb_mon_bfm);
end
```

Putting it on the HDL side is more elegant as the *uvm_config_db::set* calls are localized with the BFM interfaces and relative instance paths can thus be used. Additionally, full instance paths can be computed as strings using the SystemVerilog *%m* string formatter. For instance, one could use *$sformatf("%m.wb_drv_bfm")* instead of *"WB_DRV_BFM"* as the (guaranteed unique) uvm_config_db lookup key for the driver BFM virtual interface.

HVL/TB top level module

The top level testbench module includes an initial block to call the UVM *run_test()* function to launch a test, as shown below. Note that the leading package imports *uvm_pkg::run_test* and *tests_pkg::** are necessary to compile this function call.

```
module top_mac_hvl;
  import uvm_pkg::run_test;
  import tests_pkg::*;

  initial
    run_test();  // create and start running test

  // Optionally the initial block from above for uvm_config_db
  // registration of the HDL side BFM interfaces

endmodule
```

Simulating a Dual Top Testbench

The command line for invoking Questa vsim simply specifies all required top level modules by their module names:

```
#Makefile
...
normal: clean cmp
      vsim -c top_mac_hvl top_mac_hdl
+UVM_TESTNAME=test_mac_simple_duplex -do "run -all; exit"
...
```